PARTE 4 DE 4 · 4-5 MINUTOS

Isaac

API REST, validaciones, errores y demo en vivo

Cierras la exposición. Muestras **cómo se usa la API**: controladores, DTOs, @Valid y el manejo global de errores. Luego haces una **demo en vivo en Postman** que prueba lo que explicaron tus compañeros, y terminas con la conclusión del grupo.

Orden de la exposición

#	Quién	Tema	Tiempo
1	Santiago Arango	Introducción, tecnologías y arquitectura	3 min
2	Alex Monroy	Modelo de datos: entidades, relaciones y repositorios	3 min
3	Jose Vasquez	Lógica de negocio: servicios, reglas y excepciones	3-4 min
4	Isaac	API REST, validaciones, errores y demo en vivo	4-5 min

Antes de empezar

- Aplicación corriendo en **http://localhost:8888**, probada antes de entrar.
- Base de datos **vacía**, para que los ids de la demo sean 1 y 2.
- Postman con las peticiones de la demo guardadas en orden.
- Abiertos: **StudentController.java**, **CreateStudentDto.java** y **GlobalExceptionHandler.java**.

Guion paso a paso

Cada paso dice qué mostrar en pantalla y qué decir. No lo leas palabra por palabra: úsalo como guía y dilo con tus palabras.

1. Endpoints

Muestra: StudentController.java

```
@RestController
@RequestMapping("/students")
public class StudentController {
    @PostMapping
    public ResponseEntity<StudentResponseDto> create(
            @Valid @RequestBody CreateStudentDto dto) { ... }
}
```

QUÉ DECIR

Hay tres controladores, uno por recurso, cada uno con el CRUD completo: listar, consultar, crear, actualizar y eliminar. Con ResponseEntity devolvemos el código correcto: 201 al crear y 204 al eliminar.

2. DTOs y @Valid

Muestra: CreateStudentDto.java

```
public record CreateStudentDto(
        @NotBlank String firstName,
        @NotBlank @Email String email,
        @NotNull @Past LocalDate birthDate, ...
) { ... }
```

QUÉ DECIR

No recibimos ni devolvemos entidades, sino DTOs: así controlamos qué datos entran y salen, y evitamos ciclos infinitos al convertir a JSON. En los DTOs están las reglas de formato, y con @Valid Spring las revisa antes de entrar al método; si algo falla, responde 400 diciendo qué campo está mal.

3. Manejo global de errores

Muestra: GlobalExceptionHandler.java

QUÉ DECIR

Esta clase atrapa en un solo lugar las excepciones que lanzan los servicios y las convierte en respuestas HTTP: 404 si algo no existe, 409 si hay un conflicto como un curso lleno, y 400 si los datos están mal. Siempre con el mismo formato, un mensaje y la hora.

4. Demo en vivo en Postman

Muestra: Postman. Antes de enviar cada petición, di qué esperas que responda.

```
POST /students    { "firstName": "Juan", "lastName": "Pérez", "email": "juan@correo.com",
                    "phone": "3001234567", "birthDate": "2002-05-10" }
POST /courses     { "code": "BACK-II", "name": "Backend II", "maxCapacity": 1 }
POST /enrollments { "studentId": 1, "courseId": 1 }
PUT  /enrollments/1  { "status": "CANCELLED" }
```

#	Petición	Esperado	Qué demuestra
1	POST /students (Juan) y (María)	201	Crear estudiantes
2	POST /courses con maxCapacity: 1	201	Curso con un solo cupo
3	POST /enrollments (Juan, curso 1)	201	Matrícula correcta
4	POST /enrollments (María, curso 1)	409	Curso sin cupo
5	PUT /enrollments/1 { status: CANCELLED }	200	Cambio de estado
6	POST /enrollments (María, curso 1)	201	Se liberó el cupo
7	POST /students con email: "juan"	400	Validación con @Valid
8	GET /students/999	404	Recurso inexistente

QUÉ DECIR

Creo dos estudiantes y un curso con un solo cupo. Matriculo a Juan y funciona. Si intento con María, 409 porque el curso está lleno. Cancelo la matrícula de Juan, se libera el cupo y ahora María sí entra. Y los errores: un correo mal escrito da 400 y un id que no existe da 404.

5. Conclusión del grupo

QUÉ DECIR

Construimos una API REST con Spring Boot y MySQL, organizada por capas, con reglas de negocio y manejo de errores ordenado. Aprendimos a modelar relaciones con JPA y a validar datos. Como mejoras a futuro pensamos en pruebas automáticas, autenticación y paginación. Muchas gracias.

Preguntas que te pueden hacer

¿Por qué no devolver la entidad directamente?

Expone datos internos, acopla la API a la base de datos y con relaciones bidireccionales genera ciclos infinitos en el JSON.

¿Diferencia entre 400, 404 y 409?

400: datos inválidos. 404: no existe. 409: datos válidos pero chocan con lo que ya hay, como un curso lleno.

¿Qué pasa sin @Valid?

Las anotaciones del DTO no se revisan y el error saldría como un 500 poco claro.

Cierre

QUÉ DECIR

Con esto terminamos la exposición. ¿Tienen alguna pregunta?

Consejos

- Si una pregunta es de la parte de otro compañero, pásasela: "Eso lo explicó Alex, él les puede responder".
- Si algo falla en vivo, no te bloquees: di qué debería responder y sigue con la siguiente petición.
- Ensayen una vez los cuatro seguidos con cronómetro. Si alguien se pasa, el que sufre es la demo del final.
- Cuando muestren código, hagan zoom en IntelliJ (Ctrl + rueda del mouse) para que se lea desde atrás.
- Si no saben una respuesta, digan qué parte del código la resuelve o que lo revisan; es mejor que inventar.